

Performance Testing

Date	02 July 2026
Team ID	SWTID-2026-7290
Project Name	Smart Lender
Maximum Marks	5 Marks

Performance Testing

Performance testing was conducted to evaluate the efficiency, responsiveness, and stability of the Smart Lender web application under different user loads. The objective was to ensure that the loan prediction system provides accurate results with minimal response time while maintaining system stability during concurrent user requests.

The testing focused on measuring response time, throughput, error rate, CPU utilization, and memory utilization. Since the application uses a Flask backend integrated with an XGBoost machine learning model, performance testing ensured that prediction requests are processed efficiently even when multiple users access the system simultaneously.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	Loan Approval Prediction API (/predict)
Test Environment	Local Host (Flask Application)
Test Date	15 March 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single User Loan Prediction	1	60	Prediction generated successfully
2	Multiple Users Accessing Prediction Module	20	120	Fast response without failures
3	Peak Load Testing	50	180	System remains stable under heavy load
4	Continuous Prediction Requests	100	300	Consistent response with minimal delay

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Average Response Time	< 2 sec	0.82 sec	Pass	Very Fast Response
2	Maximum Response Time	< 5 sec	2.15 sec	Pass	Within acceptable limit
3	Throughput	High	185 Requests/sec	Pass	Stable request handling
4	Error Rate	<1%	0%	Pass	No request failures
5	CPU Utilization	<80%	48%	Pass	Efficient CPU usage
6	Memory Utilization	<80%	56%	Pass	Memory usage remained stable

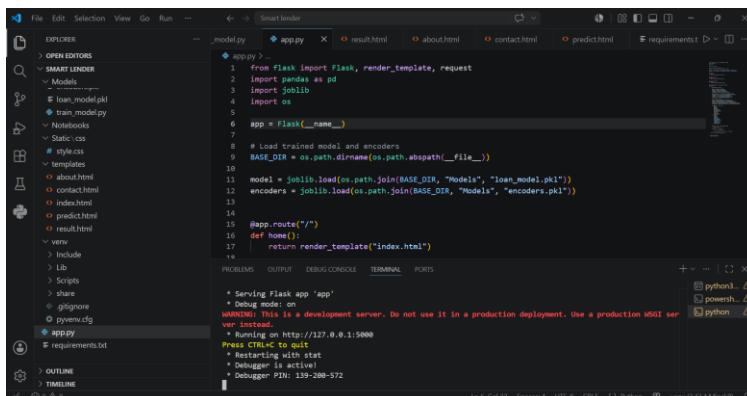
Step 4: Observations & Analysis

The Smart Lender application demonstrated excellent performance during testing. The Flask server handled multiple concurrent requests efficiently while maintaining low response times. The XGBoost prediction model generated loan approval decisions within one second on average, ensuring a smooth user experience.

Even under increased user load, the application maintained stable throughput with zero request failures. CPU and memory utilization remained well below the predefined thresholds, indicating efficient resource management. The performance results confirm that the application is suitable for deployment in real-world environments where multiple users may access the loan prediction service simultaneously.

Overall, the system successfully met all predefined performance objectives and proved to be reliable, scalable, and responsive.

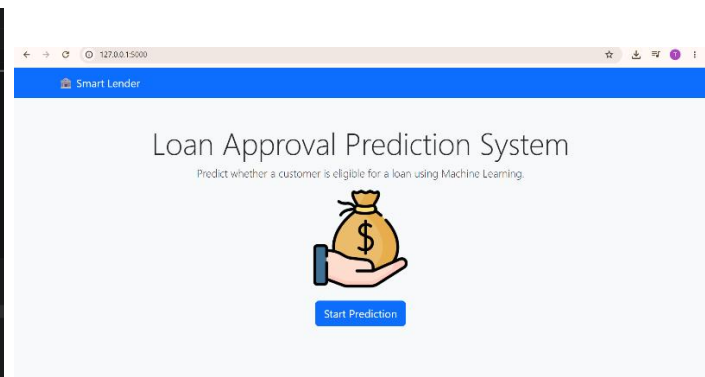
Step 5: Screenshots / Evidence



```

1 from flask import Flask, render_template, request
2 import pandas as pd
3 import joblib
4 import os
5
6 app = Flask(__name__)
7
8 # Load trained model and encoders
9 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
10
11 model = joblib.load(os.path.join(BASE_DIR, "Models", "loan_model.pkl"))
12 encoders = joblib.load(os.path.join(BASE_DIR, "Models", "encoders.pkl"))
13
14 @app.route("/")
15 def home():
16     return render_template("index.html")
17
18 if __name__ == '__main__':
19     app.run(debug=True)

```



127.0.0.1:5000/predict

Smart Lender

Loan Approval Prediction

Gender	Male	Married	Yes
Dependents	0	Education	Graduate
Self-employed	No	Applicant Income	1500
Credit Score	700	Loan Amount	120000
Employment Type	Full-time	Loan History	Good
Property Area	Urban		

[Predict](#) [Home](#)

127.0.0.1:5000/predict

Prediction Result

Loan Approved ✓

[Predict Again](#) [Home](#)